

SISTEMA DE GERENCIAMENTO DE EVENTOS ACADÊMICOS

Primeira Entrega — Concepção Técnica + MVP Inicial

Grupo:

Igor Philipo

João Vinicius

Kamylle da Silva

Roselí Maria

Orientador: Victor Henrique

Histórico de Revisão

Versão	Data	Alteração realizada	Responsável
1.0	Setembro/2026	Criação da documentação inicial do projeto, incluindo contexto do problema, requisitos funcionais (RF01–RF26), requisitos não funcionais (RNF01–RNF16) e perfis de usuário.	Grupo
1.0	Setembro/2026	Inclusão das regras de negócio (RN01–RN16), das 6 entidades do sistema e do modelo de dados (DER).	Grupo
1.0	Setembro/2026	Adição do dicionário de dados, dos contratos de entrada e saída da API, dos status codes e do padrão de resposta e erro.	Grupo
1.0	Setembro/2026	Inclusão da matriz de permissões, da estrutura inicial do projeto, das tecnologias escolhidas e da estratégia de banco de dados e migrations.	Grupo

1.0	Setembro/2026	Adição do backlog (US01–US21), dos critérios de aceitação (CA01–CA10) e do protótipo inicial da API (MVP técnico com Swagger).	Grupo
1.0	Setembro/2026	Inclusão do repositório GitHub (Seção 19) e das considerações finais (Seção 20).	Grupo

Índice

1. Contexto do Problema
2. Requisitos Funcionais
3. Requisitos Não Funcionais
4. Definição dos Perfis de Usuário
5. Regras de Negócio
6. Entidades Principais do Sistema
7. Modelo de Dados — DER
8. Dicionário de Dados
9. Contratos de Entrada e Saída da API
10. Definição dos Status Codes
11. Padrão de Resposta e Erro
12. Matriz de Permissões
13. Estrutura Inicial do Projeto
14. Tecnologias Escolhidas e Justificativa
15. Estratégia de Banco de Dados e Migrations
16. Backlog do Projeto
17. Critérios de Aceitação
18. Protótipo Inicial da API / MVP Técnico
19. Repositório GitHub
20. Considerações Finais

1. Contexto do Problema

Atualmente, o gerenciamento de eventos acadêmicos pode envolver diferentes atividades, como cadastro de eventos, divulgação de informações, controle de participantes, inscrições e emissão de certificados. Quando essas atividades são realizadas de forma manual ou utilizando diferentes ferramentas, podem ocorrer problemas como informações desorganizadas, dificuldade para acompanhar as inscrições, duplicidade de registros e demora na emissão de certificados.

Diante desse cenário, o projeto propõe o desenvolvimento de uma API REST para gerenciamento de eventos acadêmicos, centralizando as principais informações e operações relacionadas aos eventos em um único sistema.

A API permitirá que os usuários realizem operações como cadastro e consulta de eventos, gerenciamento de usuários, realização de inscrições e controle dos certificados. Dessa forma, o sistema busca tornar o processo mais organizado, seguro e eficiente.

1.1 Problema que o sistema resolve

O sistema busca solucionar a dificuldade de organizar e controlar informações relacionadas aos eventos acadêmicos. A utilização de processos manuais pode vir a dificultar o acompanhamento dos eventos, dos participantes inscritos e dos certificados emitidos.

Com a API, essas informações podem ser armazenadas e gerenciadas de forma centralizada, permitindo assim um maior controle sobre os dados e facilitando o acesso às informações.

1.2 Usuários do sistema

A primeira versão do sistema será planejada para atender os três principais perfis de usuários:

- **Administrador:** responsável pelo gerenciamento geral do sistema, incluindo usuários e eventos.
- **Organizador:** responsável pela criação e gerenciamento dos eventos acadêmicos e pelo acompanhamento das inscrições.
- **Participante:** usuário que poderá consultar os eventos disponíveis, realizar inscrições e acessar seus certificados.

Os perfis serão definidos desde a primeira versão para que o sistema possa receber posteriormente mecanismos de autenticação e autorização, como login e controle de acesso por perfil.

1.3 Objetivo da API

O objetivo da API é disponibilizar uma estrutura centralizada para o gerenciamento de eventos acadêmicos, permitindo assim que diferentes aplicações possam utilizar seus recursos por meio de requisições HTTP.

A API deverá possibilitar, inicialmente:

- cadastrar usuários;
- consultar usuários;
- cadastrar eventos;
- consultar eventos;
- atualizar informações dos eventos;
- excluir eventos conforme as permissões definidas;
- realizar inscrições em eventos;
- consultar inscrições;
- controlar certificados;
- validar os dados recebidos;
- disponibilizar a documentação das rotas por meio do Swagger.

1.4 Escopo da primeira versão

A primeira versão terá como foco a construção da base técnica do sistema e a implementação das principais operações de gerenciamento.

O MVP inicial contemplará:

- criação do projeto utilizando FastAPI;
- organização da estrutura de pastas;
- criação dos modelos das principais entidades;
- definição dos schemas para entrada e saída de dados;
- conexão com banco de dados;
- criação das principais rotas da API;
- implementação de operações CRUD das entidades definidas;
- validação dos dados utilizando Pydantic;
- documentação automática da API por meio do Swagger;
- criação do repositório do projeto no GitHub.

Recursos mais avançados, como autenticação utilizando JWT, controle efetivo de permissões, testes automatizados e outras funcionalidades complementares poderão ser desenvolvidos nas etapas seguintes do projeto.

1.5 Resultado esperado

Ao final da primeira entrega, espera-se que o projeto possua uma API funcional capaz de realizar as principais operações de gerenciamento de eventos acadêmicos, com banco de dados estruturado, rotas organizadas, validação de informações e documentação disponível no Swagger.

A primeira versão servirá como base para a evolução do sistema nas próximas etapas, permitindo posteriormente a implementação de recursos de segurança, autenticação, autorização e testes automatizados.

2. Requisitos Funcionais (RF)

ID	Descrição do Requisito Funcional	Prioridade	Requisitos Relacionados
RF-01	Cadastrar Usuário – Permitir o cadastro de novos usuários informando nome, e-mail, senha e perfil de acesso.	Essencial	RF-02, RF-03, RF-04, RF-07, RF-10, RF-11, RF-14, RF-15, RF-16, RF-17, RF-18, RF-19, RF-20, RF-21, RF-22, RF-23, RF-24, RF-25, RF-26
RF-02	Consultar Usuário – Permitir a consulta dos usuários cadastrados no sistema.	Essencial	RF-01, RF-03
RF-03	Atualizar Usuário – Permitir a atualização dos dados de um usuário cadastrado.	Essencial	RF-01, RF-02

RF-04	Realizar Login – Permitir que os usuários cadastrados realizem autenticação no sistema.	Essencial	RF-01, RF-07, RF-10, RF-11, RF-14, RF-15, RF-16, RF-17, RF-18, RF-19, RF-20, RF-21, RF-22, RF-23, RF-24, RF-25, RF-26
RF-05	Cadastrar Categoria – Permitir o cadastro de categorias para classificação dos eventos acadêmicos.	Essencial	RF-06, RF-07, RF-08, RF-09, RF-10, RF-11, RF-12, RF-13, RF-14, RF-17, RF-18
RF-06	Consultar Categorias – Permitir a consulta das categorias cadastradas no sistema.	Essencial	RF-05, RF-07, RF-08, RF-09, RF-10, RF-11, RF-12, RF-13, RF-14, RF-17, RF-18
RF-07	Cadastrar Evento – Permitir que usuários autorizados cadastrem eventos acadêmicos informando título, descrição, data, horário, local, capacidade e categoria.	Essencial	RF-01, RF-04, RF-05, RF-06, RF-08, RF-09, RF-10, RF-11, RF-12, RF-13, RF-14, RF-17, RF-18, RF-19, RF-20, RF-21, RF-24, RF-25, RF-26
RF-08	Consultar Eventos – Permitir consultar os eventos cadastrados no sistema.	Essencial	RF-05, RF-06, RF-07, RF-09, RF-10, RF-11, RF-12, RF-13, RF-14, RF-17, RF-18, RF-24, RF-25, RF-26

RF-09	Visualizar Detalhes do Evento – Permitir visualizar informações detalhadas de um evento específico.	Essencial	RF-07, RF-08, RF-10, RF-11, RF-12, RF-13, RF-14, RF-17, RF-18, RF-19, RF-20, RF-21, RF-24, RF-25, RF-26
RF-10	Atualizar Evento – Permitir que usuários autorizados atualizem os dados de um evento existente.	Essencial	RF-01, RF-04, RF-07, RF-08, RF-09
RF-11	Excluir Evento – Permitir que usuários autorizados excluam eventos cadastrados.	Essencial	RF-01, RF-04, RF-07, RF-08, RF-09
RF-12	Buscar Eventos – Permitir pesquisar eventos pelo título ou nome.	Essencial	RF-05, RF-06, RF-07, RF-08, RF-09
RF-13	Filtrar Eventos – Permitir filtrar eventos por categoria e/ou data.	Essencial	RF-05, RF-06, RF-07, RF-08, RF-09
RF-14	Realizar Inscrição – Permitir que os participantes realizem inscrições em eventos acadêmicos.	Essencial	RF-01, RF-04, RF-07, RF-08, RF-09, RF-15, RF-16, RF-17, RF-18, RF-19, RF-20

RF-15	Consultar Inscrição – Permitir que o participante consulte suas inscrições.	Essencial	RF-01, RF-04, RF-14, RF-16, RF-19, RF-20
RF-16	Cancelar Inscrição – Permitir que o participante cancele sua própria inscrição.	Essencial	RF-01, RF-04, RF-14, RF-15
RF-17	Consultar Participantes – Permitir que usuários autorizados consultem os participantes inscritos em um evento.	Essencial	RF-01, RF-04, RF-05, RF-06, RF-07, RF-08, RF-09, RF-14, RF-18
RF-18	Controlar Capacidade – Permitir controlar a quantidade máxima de participantes de cada evento.	Essencial	RF-01, RF-04, RF-05, RF-06, RF-07, RF-08, RF-09, RF-14, RF-17
RF-19	Emitir Certificado – Permitir a emissão de certificados para participantes que atendam aos critérios definidos.	Essencial	RF-01, RF-04, RF-07, RF-09, RF-14, RF-15, RF-20, RF-21
RF-20	Consultar Certificado – Permitir que o participante consulte seus certificados.	Essencial	RF-01, RF-04, RF-14, RF-15, RF-19, RF-21

RF-21	Validar Certificado – Permitir validar certificados por meio de um código de validação.	Essencial	RF-01, RF-04, RF-07, RF-09, RF-19, RF-20
RF-22	Editar Perfil – Permitir que o usuário atualize seus próprios dados.	Essencial	RF-01, RF-04
RF-23	Alterar Senha – Permitir que o usuário altere sua senha.	Essencial	RF-01, RF-04
RF-24	Favoritar Evento – Permitir que o participante marque um evento como favorito.	Essencial	RF-01, RF-04, RF-07, RF-08, RF-09, RF-25, RF-26
RF-25	Consultar Eventos Favoritos – Permitir que o participante consulte seus eventos favoritos.	Essencial	RF-01, RF-04, RF-07, RF-08, RF-09, RF-24, RF-26
RF-26	Remover Evento dos Favoritos – Permitir que o participante remova um evento da sua lista de favoritos.	Essencial	RF-01, RF-04, RF-07, RF-08, RF-09, RF-24, RF-25

3. Requisitos não Funcionais (RNF)

ID	Requisito Não Funcional	Descrição
RNF01	Desempenho	O sistema deverá apresentar respostas adequadas para as operações realizadas pela API.
RNF02	Segurança	O sistema deverá proteger os dados armazenados e impedir acessos não autorizados.
RNF03	Autenticação	O sistema deverá possuir mecanismo de autenticação dos usuários.
RNF04	Autorização	O sistema deverá controlar o acesso às funcionalidades de acordo com o perfil do usuário.
RNF05	Validação de dados	Os dados enviados para a API deverão ser validados antes de serem processados.
RNF06	Integridade dos dados	O sistema deverá garantir a consistência das informações armazenadas no banco de dados.
RNF07	Organização do código	O código deverá ser organizado em módulos, facilitando sua manutenção.

RNF08	Documentação da API	A API deverá possuir documentação utilizando Swagger/OpenAPI.
RNF09	Banco de dados	O sistema deverá utilizar SQLite como banco de dados
RNF10	Migrations	As alterações na estrutura do banco deverão ser controladas por migrations.
RNF11	Tratamento de erros	A API deverá utilizar códigos HTTP adequados e mensagens claras para informar erros.
RNF12	Manutenibilidade	A estrutura do projeto deverá facilitar futuras alterações e melhorias.
RNF13	Escalabilidade	O sistema deverá permitir a inclusão de novas funcionalidades futuramente.
RNF14	Compatibilidade	A aplicação deverá funcionar em ambientes compatíveis com Python e suas dependências.
RNF15	Testabilidade	As funcionalidades deverão poder ser testadas utilizando o Swagger e posteriormente testes automatizados.

RNF16	Versionamento	O código deverá ser disponibilizado e compartilhado por meio do GitHub, facilitando assim a colaboração entre os integrantes do grupo.
--------------	---------------	--

4. Definição dos Perfis de Usuário

O sistema de gerenciamento de eventos acadêmicos será utilizado por diferentes tipos de usuários, cada um com responsabilidades e permissões específicas. A definição desses perfis é muito importante para organizar o acesso às funcionalidades do sistema e estabelecer quais operações cada usuário poderá realizar.

Na primeira entrega, os perfis serão definidos como parte do planejamento do sistema. O controle efetivo de autenticação e autorização será implementado nas etapas posteriores do projeto.

4.1 Administrador

O administrador será o responsável pelo gerenciamento geral do sistema.

Entre suas principais responsabilidades estão:

- cadastrar, consultar, atualizar e gerenciar usuários;
- cadastrar e gerenciar eventos;
- consultar inscrições;
- acompanhar os participantes dos eventos;
- gerenciar certificados;
- excluir registros quando possuir autorização;
- administrar as informações gerais do sistema.

O Administrador terá o maior nível de acesso dentro da aplicação, podendo executar operações administrativas que não estarão disponíveis para os demais perfis.

4.2 Organizador

O organizador ficará responsável pelo gerenciamento dos eventos acadêmicos sob sua responsabilidade.

Entre suas principais funções estão:

- cadastrar eventos;
- atualizar informações dos eventos;
- consultar eventos;
- acompanhar as inscrições;
- consultar a lista de participantes;
- auxiliar no gerenciamento dos certificados dos participantes.

O Organizador não terá acesso às funções administrativas gerais do sistema, ficando limitado às funcionalidades relacionadas aos eventos que estiver autorizado a gerenciar.

4.3 Participante

O participante será o usuário interessado em participar dos eventos acadêmicos disponíveis no sistema.

Entre suas principais funções estão:

- consultar eventos disponíveis;
- visualizar informações dos eventos;
- realizar inscrições;
- consultar suas próprias inscrições;
- cancelar sua inscrição quando permitido;
- consultar seus certificados.

O Participante não poderá criar ou excluir eventos e não terá acesso às informações administrativas de outros usuários.

5. Regras de Negócio

ID	Regra de Negócio	Descrição
RN01	E-mail único	O e-mail de cada um dos usuários deverão ser único.
RN02	Inscrição única por evento	Um usuário não poderá realizar duas inscrições ativas no mesmo evento.
RN03	Limite de capacidade	A quantidade de participantes não poderá ultrapassar a capacidade do evento.
RN04	Data do evento	Um evento não poderá ser cadastrado com uma data anterior à data atual.
RN05	Permissão para cadastro	Somente os usuários autorizados poderão cadastrar eventos.
RN06	Gestão do evento	O organizador poderá alterar apenas os eventos que estão sob sua responsabilidade.
RN07	Permissão para exclusão	Somente os usuários autorizados poderão excluir eventos.
RN08	Cancelamento de inscrição	O participante poderá apenas cancelar as suas próprias inscrições.

RN09	Vínculo do certificado	O certificado deverá estar relacionado a uma inscrição válida.
RN10	Validação de certificado	Cada certificado deverá possuir um código único de validação.
RN11	Ocultação de senha	A senha do usuário não deverá ser apresentada nas respostas da API.
RN12	Categoria obrigatória	Todo evento deverá possuir uma categoria válida.
RN13	Capacidade mínima	A capacidade do evento deverá ser maior que zero.
RN14	Critério de emissão	Somente os participantes que cumprirem os critérios definidos poderão receber certificado.
RN15	Favorito único	O participante não poderá adicionar o mesmo evento aos favoritos mais de uma vez.
RN16	Remoção de favoritos	O participante poderá remover somente os eventos que ele mesmo adicionar aos favoritos.

6. ENTIDADES PRINCIPAIS DO SISTEMA

Para estruturar o sistema de gerenciamento de eventos acadêmicos, foram definidas as principais entidades responsáveis por armazenar e organizar as informações utilizadas pela aplicação. Cada entidade dessa representa uma parte importante do funcionamento do sistema e possui dados relacionados às suas respectivas funções.

6.1 Usuário

A entidade Usuário ficará responsável por armazenar os dados das pessoas que utilizam o sistema, como nome, e-mail, senha e perfil de acesso. Esses dados permitem a identificação de cada usuário e controlar suas funcionalidades dentro da aplicação.

6.2 Categoria

A entidade Categoria é utilizada para organizar os eventos de acordo com seus respectivos tipos ou áreas. Dessa forma, os eventos poderão ser classificados e posteriormente localizados de maneira mais fácil pelos participantes.

6.3 Evento

A entidade Evento reúne as principais informações dos eventos acadêmicos cadastrados no sistema, como título, descrição, data, horário, local, capacidade e categoria. É uma das entidades centrais da aplicação, pois concentra as informações necessárias para a divulgação e participação nos eventos.

6.4 Inscrição

A entidade Inscrição registra a participação de um usuário em determinado evento. Ela permite que identifiquem os participantes que estão inscritos em cada evento e também auxilia no controle da quantidade de vagas disponíveis.

6.5 Certificado

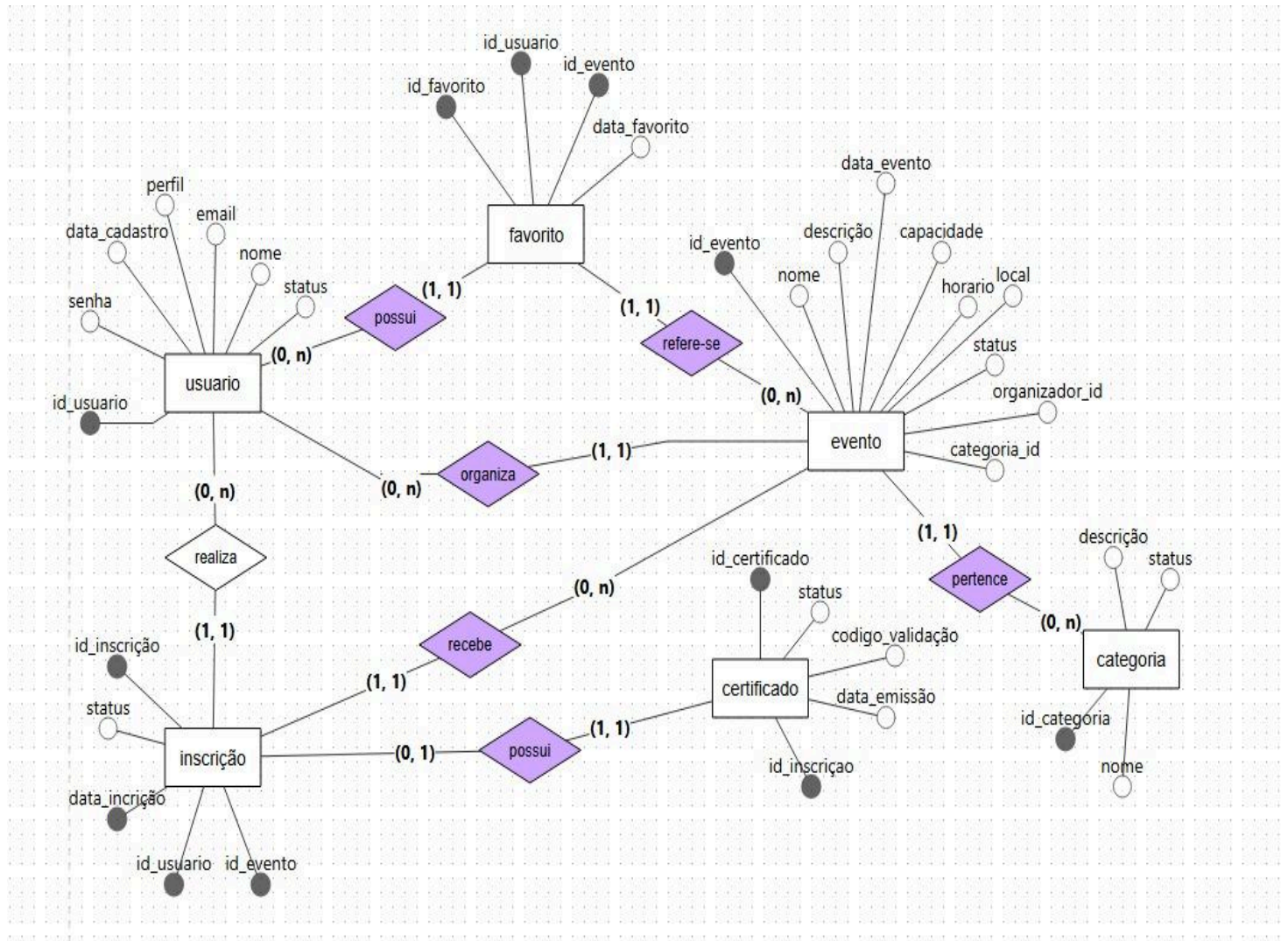
A entidade Certificado vai armazenar os certificados relacionados à participação dos usuários nos eventos. Cada certificado fica associado à inscrição correspondente, permitindo registrar e identificar a certificação concedida ao participante.

6.6 Favorito

A entidade Favorito irá registrar os eventos que foram marcados pelos participantes como favoritos. Esse recurso permite que o usuário tenha acesso mais rápido aos

eventos de seu interesse e mantenha uma lista dos eventos que deseja acompanhar.

7. Modelo de Dados — DER



8. Dicionário de Dados

8.1 Usuários

Campo	Tipo	Obrigatório	Regra
id	Integer	Sim	Chave primária
nome	Varchar	Sim	—
email	Varchar	Sim	Único e válido
senha	Varchar	Sim	Armazenada com hash
perfil	Varchar	Sim	admin/organizador/participante
data_cadastro	DateTime	Sim	Preenchido automaticamente
status	Varchar	Sim	ativo/inativo

8.2 Categorias

Campo	Tipo	Obrigatório	Regra
id	Integer	Sim	Chave primária

nome	Varchar	Sim	Único
descricao	Text	Não	—
status	Varchar	Sim	ativo/inativo

8.3 Eventos

Campo	Tipo	Obrigatório	Regra
id	Integer	Sim	Chave primária
nome	Varchar	Sim	—
descricao	Text	Não	—
data_evento	Date	Sim	Não pode ser passada
horario	Time	Sim	—
local	Varchar	Sim	—

capacidade	Integer	Sim	Maior que zero
status	Varchar	Sim	ativo/cancelado
categoria_id	Integer	Sim	Chave estrangeira
organizador_id	Integer	Sim	Chave estrangeira

8.4 Inscrição

Campo	Tipo	Obrigatório	Regra
id	Integer	Sim	Chave primária
usuario_id	Integer	Sim	Chave estrangeira
evento_id	Integer	Sim	Chave estrangeira
data_inscricao	DateTime	Sim	Preenchido automaticamente
status	Varchar	Sim	ativa/cancelada

8.5 Certificados

Campo	Tipo	Obrigatório	Regra
id	Integer	Sim	Chave primária
inscricao_id	Integer	Sim	Chave estrangeira
codigo_validacao	Varchar	Sim	Único
data_emissao	DateTime	Sim	Preenchido automaticamente
status	Varchar	Sim	emitido/cancelado

8.6 Favoritos

Campo	Tipo	Obrigatório	Regra
id	Integer	Sim	Chave primária
usuario_id	Integer	Sim	Chave estrangeira
evento_id	Integer	Sim	Chave estrangeira

data_favorito	DateTime	Sim	Preenchido automaticamente
----------------------	----------	-----	----------------------------

9. CONTRATOS DE ENTRADA E SAÍDA DA API

Os contratos definem os dados que a API recebe nas requisições e os dados que serão devolvidos nas respostas.

9.1. Cadastro de Usuário

Aqui é onde criamos a conta do usuário no sistema. Passamos os dados básicos e o perfil dele (ex: *participante* ou *organizador*).

- **Rota:** POST /usuarios
- **Corpo da Requisição (Entrada):**

```
{
  "nome": "Maria Silva",
  "email": "maria@email.com",
  "senha": "123456",
  "perfil": "participante"
}
```

Resposta (201 Created):

```
{
  "id": 1,
  "nome": "Maria Silva",
  "email": "maria@email.com",
  "perfil": "participante"
}
```

9.2. Consulta de Usuários

Esta rota retorna a lista com todos os usuários cadastrados.

- **Rota:** GET /usuarios

- **Resposta (200 OK):**

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "nome": "Maria Silva",
      "email": "maria@email.com",
      "perfil": "participante"
    },
    {
      "id": 2,
      "nome": "João Santos",
      "email": "joao@email.com",
      "perfil": "organizador"
    }
  ],
  "message": "Usuários consultados com sucesso."
}
```

9.3. Cadastro de Categoria

Utilizada para cadastrar novas categorias para os eventos (ex: Tecnologia, Negócios, etc.).

- **Rota:** POST /categorias
- **Corpo da Requisição (Entrada):**

```
{
  "nome": "Tecnologia",
  "descricao": "Eventos relacionados à tecnologia e programação."
}
```

Resposta (201 Created):

```
{
  "id": 1,
  "nome": "Tecnologia",
  "descricao": "Eventos relacionados à tecnologia e programação."
}
```

```
}
```

9.4. Consulta de Categorias

Busca todas as categorias ativas no sistema.

- **Rota:** GET /categorias
- **Resposta** (200 OK):

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "nome": "Tecnologia",
      "descricao": "Eventos relacionados à tecnologia e programação."
    },
    {
      "id": 2,
      "nome": "Administração",
      "descricao": "Eventos relacionados à área de administração."
    }
  ],
  "message": "Categorias consultadas com sucesso."
}
```

9.5. Cadastro de Evento

Permite a criação de um novo evento associado a uma categoria. Na resposta, a API vincula automaticamente o **organizador_id**.

- **Rota:** POST /eventos
- **Corpo da Requisição (Entrada):**

```
{
  "nome": "Workshop de Python",
  "descricao": "Introdução à programação Python.",
  "data_evento": "2026-10-10",
  "horario": "14:00",
  "local": "Laboratório 01",
  "capacidade": 50,
}
```

```
"categoria_id": 1
}
```

Resposta (201 Created):

```
{
  "id": 1,
  "nome": "Workshop de Python",
  "descricao": "Introdução à programação Python.",
  "data_evento": "2026-10-10",
  "horario": "14:00",
  "local": "Laboratório 01",
  "capacidade": 50,
  "categoria_id": 1,
  "organizador_id": 2
}
```

9.6. Listagem de Eventos

Traz uma visão geral dos eventos disponíveis no sistema.

- **Rota:** GET /eventos
- **Resposta (200 OK):**

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "nome": "Workshop de Python",
      "data_evento": "2026-10-10",
      "horario": "14:00",
      "local": "Laboratório 01",
      "capacidade": 50,
      "categoria_id": 1
    }
  ],
  "message": "Eventos consultados com sucesso."
}
```

9.7. Detalhes de um Evento Específico

Traz as informações completas de um evento pelo seu ID.

- **Rota:** GET /eventos/{evento_id}
- **Resposta** (200 OK):

```
{
  "id": 1,
  "nome": "Workshop de Python",
  "descricao": "Introdução à programação Python.",
  "data_evento": "2026-10-10",
  "horario": "14:00",
  "local": "Laboratório 01",
  "capacidade": 50,
  "categoria_id": 1,
  "organizador_id": 2
}
```

9.8. Atualização de Evento

Utilizada para editar as informações de um evento existente.

- **Rota:** PUT /eventos/{evento_id}
- **Corpo da Requisição (Entrada):**

```
{
  "nome": "Workshop de Python Avançado",
  "descricao": "Workshop sobre conceitos de Python.",
  "data_evento": "2026-10-15",
  "horario": "15:00",
  "local": "Laboratório 02",
  "capacidade": 60,
  "categoria_id": 1
}
```

Resposta (200 OK):

JSON

```
{
  "id": 1,
  "nome": "Workshop de Python Avançado",
```

```
"descricao": "Workshop sobre conceitos de Python.",
"data_evento": "2026-10-15",
"horario": "15:00",
"local": "Laboratório 02",
"capacidade": 60,
"categoria_id": 1,
"organizador_id": 2
}
```

9.9. Realizar Inscrição

Inscriver um usuário em determinado evento.

- **Rota:** POST /eventos/{evento_id}/inscricoes
- **Corpo da Requisição (Entrada):**

```
{
  "usuario_id": 3
}
```

Resposta (201 Created):

JSON

```
{
  "id": 1,
  "usuario_id": 3,
  "evento_id": 1,
  "data_inscricao": "2026-09-08T10:00:00",
  "status": "ativa"
}
```

9.10. Consultar Inscrições

Consulta a lista de inscrições efetuadas.

- **Rota:** GET /inscricoes
- **Resposta** (200 OK):

```
{
  "success": true,

```

```
"data": [  
  {  
    "id": 1,  
    "usuario_id": 3,  
    "evento_id": 1,  
    "data_inscricao": "2026-09-08T10:00:00",  
    "status": "ativa"  
  }  
],  
"message": "Inscrições consultadas com sucesso."  
}
```

9.11. Cancelar Inscrição

Remove ou cancela a inscrição informada no parâmetro de URL.

- **Rota:** DELETE /inscricoes/{inscricao_id}
- **Resposta (204 No Content):** *(Sem conteúdo no corpo da resposta)*

9.12. Emissão de Certificado

Gera o certificado digital para o participante após a conclusão do evento.

- **Rota:** POST /inscricoes/{inscricao_id}/certificado
- **Resposta (201 Created):**

```
{  
  "id": 1,  
  "inscricao_id": 1,  
  "codigo_validacao": "CERT-2026-0001",  
  "data_emissao": "2026-10-20T15:30:00",  
  "status": "emitido"  
}
```

9.13. Consultar Certificado

Busca as informações e a chave de validação de um certificado específico.

- **Rota:** GET /certificados/{certificado_id}
- **Resposta (200 OK):**

```
{
  "id": 1,
  "inscricao_id": 1,
  "codigo_validacao": "CERT-2026-0001",
  "data_emissao": "2026-10-20T15:30:00",
  "status": "emitido"
}
```

9.14. Favoritar Evento

Salva o evento na lista de favoritos do participante.

- **Rota:** POST /eventos/{evento_id}/favoritos
- **Resposta** (201 Created):

```
{
  "message": "Evento adicionado aos favoritos."
}
```

9.15. Consultar Favoritos

Traz a lista dos eventos favoritos pelos usuários.

- **Rota:** GET /favoritos
- **Resposta** (200 OK):

```
{
  "data": [
    {
      "id": 1,
      "evento_id": 5,
      "titulo": "Workshop de Python"
    }
  ]
}
```

9.16. Remover Favorito

Desfaz a ação de favoritar para um determinado evento.

- **Rota:** DELETE /eventos/{evento_id}/favoritos
- **Resposta** (204 No Content): (Sem conteúdo no corpo da resposta)

10. Definição dos Status Codes

Código	Significado
200	Operação realizada com sucesso
201	Registro criado
204	Operação realizada sem conteúdo
400	Requisição inválida/regra de negócio
401	Usuário não autenticado
403	Usuário sem permissão
404	Registro não encontrado
409	Conflito
422	Dados enviados inválidos

11. Padrão de Resposta e Erro

11.1 Resposta de sucesso

```
{  
  "success": true,  
  "data": {},  
  "message": "Operação realizada com sucesso."  
}
```

11.2 Resposta de Erro

```
{  
  "detail": "Evento não encontrado."  
}
```

12. Matriz de Permissões

Funcionalidade	Admin	Organizador	Participante
Cadastrar usuário	SIM	NÃO	SIM
Consultar usuários	SIM	NÃO	NÃO
Cadastrar categoria	SIM	NÃO	NÃO
Cadastrar evento	SIM	SIM	NÃO
Alterar evento	SIM	SIM	NÃO

Excluir evento	SIM	NÃO	NÃO
Consultar eventos	SIM	SIM	SIM
Buscar eventos	SIM	SIM	SIM
Filtrar eventos	SIM	SIM	SIM
Ver detalhes	SIM	SIM	SIM
Realizar inscrição	NÃO	NÃO	SIM
Cancelar própria inscrição	NÃO	NÃO	SIM
Ver participantes	SIM	SIM	NÃO
Emitir certificado	SIM	SIM	NÃO
Consultar certificado	NÃO	NÃO	SIM
Editar próprio perfil	SIM	SIM	SIM
Alterar própria senha	SIM	SIM	SIM
Favoritar evento	NÃO	NÃO	SIM

Consultar favoritos	NÃO	NÃO	SIM
Remover favorito	NÃO	NÃO	SIM

13. ESTRUTURA INICIAL DO PROJETO

O projeto será organizado da seguinte forma:

sistema-eventos/

```

|
|— app/
|   |— api/
|       |— v1/
|           |— routers/
|               |— usuarios.py
|               |— eventos.py
|               |— categorias.py
|               |— inscricoes.py
|               |— certificados.py
|               |— favoritos.py
|               |— api.py
|   |— models/
|   |— schemas/
|   |— database/
|   |— core/
|

```

- |— alembic/
- |— tests/
- |— docs/
- |
- |— main.py
- |— requirements.txt
- |— alembic.ini
- |— .env
- |— .gitignore
- └— README.md

14. TECNOLOGIAS ESCOLHIDAS E JUSTIFICATIVA

Python

Será utilizado como linguagem de programação principal, por ser uma linguagem versátil, amplamente utilizada no mercado e com grande ecossistema de bibliotecas.

FastAPI

Será utilizado para desenvolver a API REST, por ser um framework moderno, assíncrono, com validação automática de dados via Pydantic, documentação Swagger integrada e alta performance.

SQLite

Será utilizado como banco de dados, por ser leve, não exigir instalação de servidor e ser ideal para o desenvolvimento e testes da aplicação.

SQLAlchemy

Será utilizado para facilitar a comunicação entre a aplicação Python e o banco de dados, por ser o ORM mais maduro do ecossistema Python, permitindo o mapeamento objeto-relacional e facilitando a manutenção do código.

Alembic

Será utilizado para controlar as alterações do banco por meio de migrations, permitindo versionamento e evitando alterações manuais na estrutura do banco.

Uvicorn

Será utilizado para executar a aplicação FastAPI, por ser um servidor ASGI leve e rápido, recomendado para rodar aplicações FastAPI.

GitHub

Será utilizado para armazenar e compartilhar o código do projeto, facilitando a colaboração entre os integrantes do grupo.

15. ESTRATÉGIA DE BANCO DE DADOS E MIGRATIONS

O banco de dados escolhido será o SQLite, por ser leve, não exigir instalação de servidor e ser ideal para o desenvolvimento e testes da aplicação.

Inicialmente serão utilizadas as tabelas:

- usuarios
- categorias
- eventos
- inscricoes
- certificados
- favoritos

Serão utilizadas chaves primárias e estrangeiras para manter os relacionamentos entre as tabelas.

O Alembic será utilizado para controlar as alterações na estrutura do banco.

Exemplo:

```
alembic revision --autogenerate -m "criacao das tabelas iniciais"
```

Para aplicar as alterações:

alembic upgrade head

As informações de conexão com o banco deverão ser armazenadas no arquivo .env.

16. Backlog do Projeto

Código	Atividade	Prioridade
US01	Cadastrar usuário	Alta
US02	Realizar login	Alta
US03	Cadastrar categoria	Alta
US04	Cadastrar evento	Alta
US05	Consultar eventos	Alta
US06	Buscar eventos	Média
US07	Filtrar eventos	Média
US08	Visualizar detalhes do evento	Alta
US09	Atualizar evento	Alta

US10	Excluir evento	Alta
US11	Realizar inscrição	Alta
US12	Consultar inscrições	Alta
US13	Cancelar inscrição	Alta
US14	Consultar participantes	Média
US15	Emitir certificado	Média
US16	Consultar certificado	Média
US17	Editar perfil	Média
US18	Alterar senha	Média
US19	Favoritar evento	Média
US20	Consultar favoritos	Média
US21	Remover favorito	Média

17. Critérios de Aceitação

CA01 — Cadastro de usuário

- O sistema deverá aceitar somente dados válidos.
- O e-mail deverá ser único.
- A senha não deverá aparecer nas respostas.
- Dados inválidos deverão gerar erro.

CA02 — Cadastro de evento

- O título deverá ser informado.
- A data deverá ser válida.
- A capacidade deverá ser maior que zero.
- A categoria deverá existir.
- O organizador deverá existir.

CA03 — Consulta de eventos

- O sistema deverá listar os eventos.
- O usuário deverá conseguir buscar eventos.
- O usuário deverá conseguir filtrar eventos.
- O usuário deverá conseguir visualizar os detalhes.

CA04 — Inscrição

- O participante deverá estar cadastrado.
- O evento deverá existir.
- Não deverá existir inscrição ativa duplicada.
- A capacidade máxima não poderá ser ultrapassada.

CA05 — Cancelamento

- O participante poderá cancelar sua própria inscrição.
- Uma inscrição inexistente deverá gerar erro.

CA06 — Certificado

- O certificado deverá estar relacionado a uma inscrição válida.
- O código deverá ser único.
- O participante deverá conseguir consultar o certificado.

CA07 — Favoritos

- O participante deverá conseguir favoritar um evento.
- O mesmo evento não poderá ser favoritado duas vezes pelo mesmo participante.
- O participante deverá conseguir consultar seus favoritos.
- O participante deverá conseguir remover um evento dos favoritos.

CA08 — Atualização de evento

- O evento deverá existir.
- O usuário deverá ter permissão.
- Os dados atualizados deverão ser válidos.
- A data não poderá ser passada. **CA09 — Exclusão de evento**

- Apenas o Admin poderá excluir.
- O evento deverá existir.
- Deverá retornar 204 em caso de sucesso.
- Deverá retornar 404 se não encontrado.

CA10 — Login / Autenticação

- O e-mail deverá estar cadastrado.
- A senha deverá conferir.
- Deverá retornar ao token JWT.
- Deverá retornar 401 em credenciais inválidas.

18. Protótipo Inicial da API / MVP Técnico

O MVP técnico foi desenvolvido utilizando FastAPI, SQLAlchemy, SQLite e Alembic, com 26 rotas implementadas (RF01 a RF26). A API possui documentação automática no padrão OpenAPI/Swagger, validação de dados com Pydantic e autenticação via JWT.

18.1 Documentação Swagger

A documentação automática da API está disponível em <http://localhost:8000/docs>, gerada pelo FastAPI:

Sistema de Gerenciamento de Eventos Acadêmicos2.0.0OAS 3.1

/openapi.json

RF01 a RF26 | FastAPI + SQLAlchemy + Alembic + SQLite | Swagger | Campo evento.nome conforme DER

Authorize

Usuários

GET/api/v1/usuariosRF02 - Consultar usuários

POST/api/v1/usuariosRF01 - Cadastrar usuário

GET/api/v1/usuarios/meRF22 - Ver próprio perfil

PATCH/api/v1/usuarios/meRF22 - Editar próprio perfil

PUT/api/v1/usuarios/me/senhaRF23 - Alterar própria senha

GET/api/v1/usuarios/{usuario_id}RF02 - Detalhar usuário

PUT/api/v1/usuarios/{usuario_id}RF03 - Atualizar usuário

POST/api/v1/usuarios/loginRF04 - Realizar login

Autenticação

POST/api/v1/auth/loginRF04 - Login global

Categorias

GET/api/v1/categoriasRF06 - Consultar categorias

POST/api/v1/categoriasRF05 - Cadastrar categoria

GET/api/v1/categorias/{categoria_id}RF06 - Detalhar categoria

Eventos

POST/api/v1/eventosRF07 - Cadastrar evento

GET/api/v1/eventosRF08 (+RF12 busca / RF13 filtro)

GET/api/v1/eventos/{evento_id}RF09 - Detalhar evento

PUT/api/v1/eventos/{evento_id}RF10 - Atualizar evento

DELETE/api/v1/eventos/{evento_id}RF11 - Excluir evento (só admin)

GET/api/v1/eventos/{evento_id}/participantesRF17 - Consultar participantes do evento

Inscrições			^
POST	/api/v1/eventos/{evento_id}/inscricoes	RF14 - Realizar inscrição	🔒 ↓
GET	/api/v1/inscricoes	RF15 - Consultar inscrições	🔒 ↓
DELETE	/api/v1/inscricoes/{inscricao_id}	RF16 - Cancelar inscrição	🔒 ↓
Certificados			^
POST	/api/v1/inscricoes/{inscricao_id}/certificado	RF19 - Emitir certificado	🔒 ↓
GET	/api/v1/certificados/validar/{codigo}	RF21 - Validar certificado por código	↓
GET	/api/v1/certificados/{certificado_id}	RF20 - Consultar certificado	🔒 ↓
Favoritos			^
POST	/api/v1/eventos/{evento_id}/favoritos	RF24 - Favoritar evento	🔒 ↓
DELETE	/api/v1/eventos/{evento_id}/favoritos	RF26 - Remover favorito	🔒 ↓
GET	/api/v1/favoritos	RF25 - Consultar favoritos	🔒 ↓
GET	/api/v1/inscricoes	RF15 - Consultar inscrições	🔒 ↓
DELETE	/api/v1/inscricoes/{inscricao_id}	RF16 - Cancelar inscrição	🔒 ↓
Certificados			^
POST	/api/v1/inscricoes/{inscricao_id}/certificado	RF19 - Emitir certificado	🔒 ↓
GET	/api/v1/certificados/validar/{codigo}	RF21 - Validar certificado por código	↓
GET	/api/v1/certificados/{certificado_id}	RF20 - Consultar certificado	🔒 ↓
Favoritos			^
POST	/api/v1/eventos/{evento_id}/favoritos	RF24 - Favoritar evento	🔒 ↓
DELETE	/api/v1/eventos/{evento_id}/favoritos	RF26 - Remover favorito	🔒 ↓
GET	/api/v1/favoritos	RF25 - Consultar favoritos	🔒 ↓
Health			^
GET	/	Health	↓
GET	/health	Health Check	↓

Documentação Swagger gerada automaticamente pelo FastAPI, com as rotas organizadas por recurso (RNF08).

18.2 Cadastro de Usuário (RF01)

A rota POST /api/v1/usuarios permite o cadastro de novos usuários com validação dos campos obrigatórios:

The screenshot shows a REST client interface with the following details:

- Request:** curl -X 'POST' \ 'http://127.0.0.1:8000/api/v1/usuarios' \ -H 'accept: */*' \ -H 'Content-Type: application/json' \ -d '{ "nome": "Maria Silva", "email": "maria3@email.com", "senha": "123456", "perfil": "participante" }'
- Request URL:** http://127.0.0.1:8000/api/v1/usuarios
- Server response:** 201
- Response body:** { "id": 6, "nome": "Maria Silva", "email": "maria3@email.com", "perfil": "participante", "data_cadastro": "2026-10-03T01:03:51.815563", "status": "ativo" }
- Response headers:** content-length: 142, content-type: application/json, date: Sat, 03 Oct 2026 01:03:59 GMT, server: unicorn
- Responses table:**

Code	Description	Links
201	Created	

Cadastro de usuário retornando 201 Created, com a senha oculta na resposta (RF01, RN11).

18.3 Autenticação via JWT (RF04)

A rota POST /api/v1/usuarios/login autentica o usuário e retorna um token JWT que será usado nas rotas protegidas:

The screenshot shows a REST client interface with the following details:

- Request:** curl -X 'POST' \ 'http://127.0.0.1:8000/api/v1/usuarios/login' \ -H 'accept: */*' \ -H 'Content-Type: application/json' \ -d '{ "email": "maria3@email.com", "senha": "123456" }'
- Request URL:** http://127.0.0.1:8000/api/v1/usuarios/login
- Server response:** 200
- Response body:** { "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1Iiwic2NvcyI6IjoicG9ydG1jaXBhbnR1IiwiaWF0IjE7eXpYTNA2MlhaaWwY29tIiwiaXNwIjoxNzkuOTkxNTY0FQ.35m1IS1cPw_848h2U0Vn9hXUz12kP2T", "token_type": "bearer", "usuario": { "id": 6, "nome": "Maria Silva", "email": "maria3@email.com", "perfil": "participante", "data_cadastro": "2026-10-03T01:03:51.815563", "status": "ativo" } }
- Response headers:** content-length: 381, content-type: application/json, date: Sat, 03 Oct 2026 01:09:23 GMT, server: unicorn
- Responses table:**

Code	Description	Links
200	Successful Response	No links

Login retornando 200 OK com o access_token JWT e os dados do usuário autenticado (RF04, CA10).

18.4 Cadastro de Categoria (RF05)

A rota POST /api/v1/categorias permite cadastrar categorias para classificação dos eventos:

The screenshot displays a REST client interface with the following details:

- Responses:** A section at the top with a light green background.
- Curl:** A code block containing the command:

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/v1/categorias' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nome": "Ciencia",
    "descricao": "Eventos de ciencia"
  }'
```
- Request URL:** A text field showing `http://127.0.0.1:8000/api/v1/categorias`.
- Server response:** A section showing the response details.
 - Code:** 201
 - Details:** A table with two columns: Code and Details.
 - Response body:** A code block showing the JSON response:

```
{
  "id": 3,
  "nome": "Ciencia",
  "descricao": "Eventos de ciencia",
  "status": "ativo"
}
```
 - Response headers:** A code block showing the headers:

```
content-length: 75
content-type: application/json
date: Sat, 03 Oct 2025 01:14:00 GMT
server: unicorn
```
- Responses Table:** A table at the bottom with columns: Code, Description, and Links.

Code	Description	Links
201	Successful Response	No links

Cadastro de categoria retornando 201 Created, com status "ativo" definido automaticamente (RF05).

18.5 Cadastro de Evento (RF07)

A rota POST /api/v1/eventos permite que usuários com perfil admin ou organizador criem eventos:



The screenshot shows a REST client interface with the following details:

- Request URL:** http://127.0.0.1:8000/api/v1/eventos
- Server response:** 201
- Response body:**

```
{
  "id": 3,
  "nome": "Workshop de Python",
  "descricao": "Introdução à programação Python.",
  "data_evento": "2026-10-10",
  "horario": "14:00:00",
  "local": "Laboratório 01",
  "capacidade": 50,
  "status": "ativo",
  "categoria_id": 1,
  "organizador_id": 7
}
```
- Response headers:**

```
content-length: 231
content-type: application/json
date: Sat, 03 Oct 2026 01:36:24 GMT
server: uvicorn
```

Cadastro de evento retornando 201 Created. O organizador_id é vinculado automaticamente ao usuário autenticado (RF07).

18.6 Listagem de Eventos (RF08)

A rota GET /api/v1/eventos retorna a lista de eventos cadastrados:



The screenshot shows a REST client interface with the following details:

- Request URL:** http://127.0.0.1:8000/api/v1/eventos
- Server response:** 200
- Response body:**

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "nome": "Evento Fav",
      "descricao": "d",
      "data_evento": "2026-10-10",
      "horario": "14:00:00",
      "local": "Lab",
      "capacidade": 50,
      "status": "ativo",
      "categoria_id": 1,
      "organizador_id": 1
    },
    {
      "id": 2,
      "nome": "Workshop de Python",
      "descricao": "Introducao a Python",
      "data_evento": "2026-12-15",
      "horario": "14:00:00",
      "local": "Laboratorio 01",
      "capacidade": 50,
      "status": "ativo",
      "categoria_id": 1,
      "organizador_id": 5
    }
  ]
}
```
- Response headers:**

```
content-length: 691
content-type: application/json
date: Sat, 03 Oct 2026 01:38:34 GMT
server: uvicorn
```

Listagem de eventos retornando 200 OK, com o padrão {success, data, message} (RF08, RNF11).

18.7 Realização de Inscrição (RF14)

A rota `POST /api/v1/eventos/{evento_id}/inscricoes` permite que participantes se inscrevam em eventos:

Responses

Curl

```
curl -X 'POST' \
'http://127.0.0.1:8000/api/v1/eventos/1/inscricoes' \
-H 'accept: */*' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI3IiwicG9VZmlsIjoib3JnYW5pemFkb3IiLCJlbWFPbCkiOiImYyZ2FuaXphZG9yQG9vcTZhbmNvbSIsImV4cCI6MTc5MDk5MzEzOjA0LmQ0LHQtODk0fQ.' \
-H 'Content-Type: application/json' \
-d '{
  "usuario_id": 6
}'
```

Request URL

```
http://127.0.0.1:8000/api/v1/eventos/1/inscricoes
```

Server response

CodeDetails

201Response body

```
{
  "id": 2,
  "usuario_id": 6,
  "evento_id": 1,
  "data_inscricao": "2026-10-03T01:43:41.103780",
  "status": "ativa"
}
```

Response headers

```
content-length: 100
content-type: application/json
date: Sat, 03 Oct 2025 01:43:40 GMT
server: uvicorn
```

Responses

CodeDescriptionLinks

Inscrição do participante no evento retornando 201 Created (RF14).

18.8 Validação de Regra de Negócio (RN01)

Ao tentar cadastrar um e-mail já existente, a API bloqueia a operação:

Responses	
Curl <pre>curl -X 'POST' \ http://127.0.0.1:8000/api/v1/usuarios' \ -H 'accept: */*' \ -H 'Content-Type: application/json' \ -d '{ "nome": "Maria Silva", "email": "maria3@email.com", "senha": "123456", "perfil": "participante" }'</pre>	
Request URL http://127.0.0.1:8000/api/v1/usuarios	
Server response Code Details 409 Error: Conflict Undocumented Response body <pre>{ "detail": "E-mail já cadastrado (RN01)."</pre> Download Response headers <pre>content-length: 42 content-type: application/json date: Sat, 03 Oct 2026 01:45:58 GMT server: uvicorn</pre>	
Responses Code Description 201 Successful Response No links	

Validação da regra RN01 (e-mail único). A API retorna 409 Conflict com a mensagem "E-mail já cadastrado (RN01)"

18.9 Banco de Dados e Migrations

O banco de dados escolhido foi o SQLite (RNF09), que é leve, não exige instalação de servidor e é ideal para o desenvolvimento e testes da aplicação. O arquivo eventos.db (~98 KB) é criado automaticamente na primeira execução da aplicação.

Foram implementadas 6 tabelas conforme o DER (Seção 7), e todas estão presentes e funcionais no banco:

Tabela	Descrição
usuarios	Armazena nome, e-mail, senha (hash) e perfil dos usuários
categorias	Classificação dos eventos
eventos	Informações dos eventos (nome, data, local, capacidade, categoria_id, organizador_id)
inscricoes	Registro de inscrições (usuario_id, evento_id, status)
certificados	Certificados emitidos (inscricao_id, codigo_validacao)
favoritos	Eventos favoritos pelos participantes

Os relacionamentos entre as tabelas são garantidos por chaves primárias e estrangeiras definidas no modelo SQLAlchemy. As alterações na estrutura do banco

são controladas via Alembic (RNF10), com a migração inicial em `alembic/versions/0001_initial.py`.

18.10 Como Executar o Projeto

1. Instalar as dependências

```
pip install -r requirements.txt
```

2. Executar o servidor

```
uvicorn main:app --reload --port 8000
```

3. Acessar a documentação Swagger

<http://localhost:8000/docs>

18.11 Tecnologias Utilizadas no MVP

Tecnologia	Uso no Projeto
Python 3.12	Linguagem principal
FastAPI	Framework da API REST
SQLAlchemy	ORM para comunicação com o banco
SQLite	Banco de dados

Alembic	Controle de migrations
Pydantic	Validação de dados
JWT	Autenticação
Uvicorn	Servidor ASGI
Swagger/OpenAPI	Documentação automática

19. Repositório GitHub

<https://github.com/17251354-collab/backend-sistema-eventos-IJKR>

20. Considerações Finais

O desenvolvimento da primeira entrega do Sistema de Gerenciamento de Eventos Acadêmicos permitiu ao grupo aplicar, na prática, os principais conceitos da disciplina de Desenvolvimento Back End. A construção da API REST com FastAPI, integrada ao SQLite via SQLAlchemy e controlada por migrations com Alembic, resultou em um sistema funcional e aderente aos requisitos definidos.

Foram implementados os 26 Requisitos Funcionais (RF01 a RF26), os Requisitos Não Funcionais (RNF01 a RNF16) e as 16 Regras de Negócio (RN01 a RN16), cobrindo cadastro, consulta, atualização e exclusão de usuários, categorias, eventos, inscrições, certificados e favoritos. A validação com Pydantic, a documentação automática via Swagger e a autenticação com JWT garantiram padronização e segurança à API.

Por fim, considera-se que a primeira entrega cumpriu os objetivos propostos, entregando uma base técnica sólida e funcional que servirá de alicerce para as próximas etapas do projeto.